(Blog Link : https://www.anthropic.com/engineering/multi-agent-research-system?ref=blog.langchain.com)

Where we are (a bridge from the last session)

Last time we covered **parallelism**. The idea there was simple:

> Take one big job → cut it into independent pieces → run all the pieces **at the same time** → stitch the results back together.

We saw three examples: two where we summarised documents (cut the pile of documents, summarise each chunk at the same time, combine), and one around RAG. In all three, the pieces were **the same kind of work** and **did not depend on each other**. That is exactly when running things in parallel pays off.

Now ask three questions that parallelism does not answer:

1. What if the pieces are **not the same kind of work** — one needs to search, one needs to write code, one needs to double-check facts?
2. What if one piece **depends on the result** of another — you can't start step B until step A tells you what it found?
3. What if you want one agent to **stay in charge**, hand out work, look at what comes back, and then decide what to do next? (Supervisor Nature)

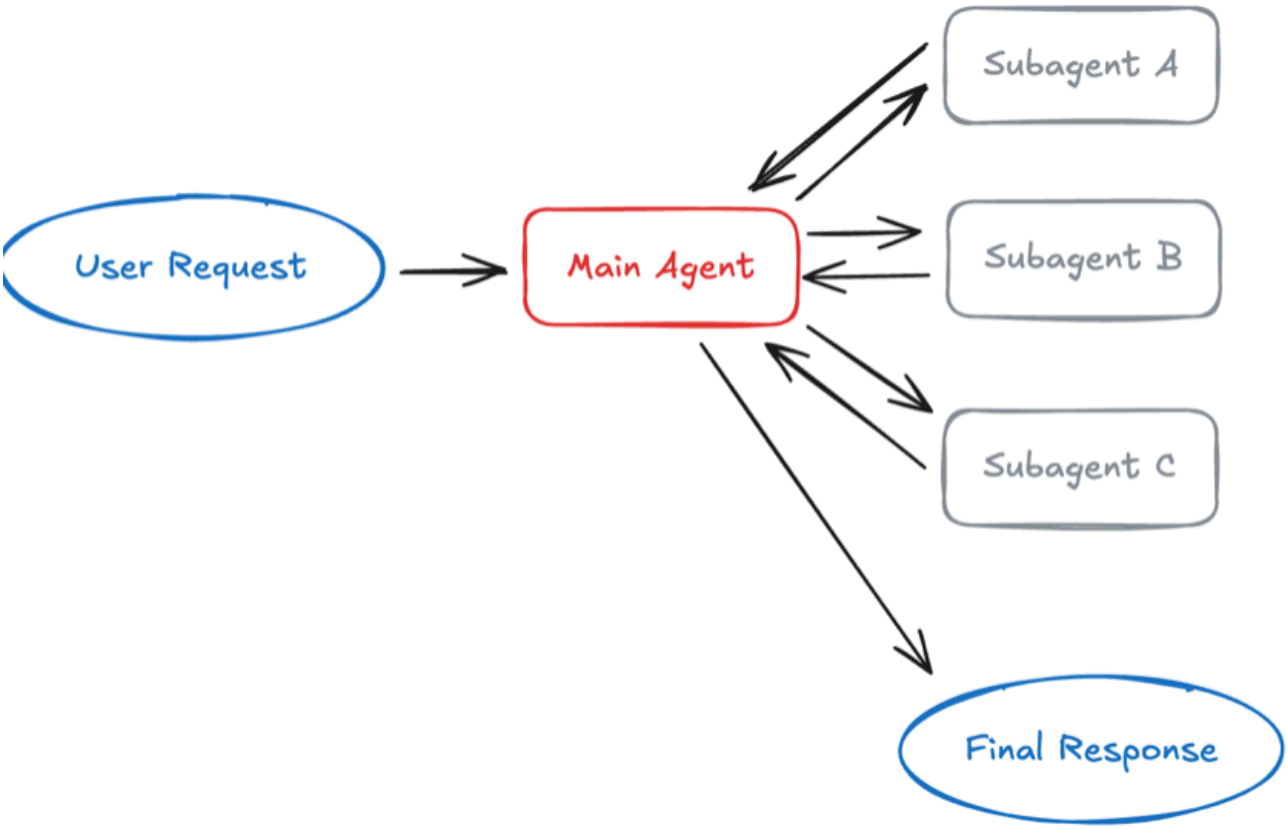
That "someone stays in charge and hands out scoped work" idea is what sub-agents are about.

1. What is a sub-agent?

A **sub-agent** is a **second agent** that a main agent calls to handle **one bounded piece of work**, and which **reports its result back**.

Two roles to name clearly:

- **Orchestrator** (also called the *parent* or *lead* agent) — the agent that owns the overall task and decides who does what.
- **Sub-agent** (the *child* or *worker*) — a focused agent that does one scoped job and returns an answer.



Many agentic tasks are best handled by a single agent with well-designed tools.

Single agents are simpler to build, reason about, and debug.

But as applications scale, teams face a common challenge wherein they have sprawling agent capabilities they want to combine into a single coherent interface.
As the features they want to combine grow in number, two main constraints emerge:

Context management: Specialised knowledge for each capability doesn't fit comfortably in a single prompt. If context windows were infinite and latency was zero, you could include all relevant information upfront. In practice, you need strategies to selectively surface information as agents work.

Distributed development: Different teams develop and maintain each capability independently, with clear boundaries and ownership. A single monolithic agent prompt becomes difficult to manage across team boundaries.

These constraints become critical when you're managing extensive domain knowledge, coordinating across teams, or tackling genuinely complex tasks. In these cases, multi-agent architectures *can* become the right choice.

Recent research demonstrates how multi-agent systems perform better in these situations. In Anthropic's multi-agent research system, a multi-agent architecture with Claude Opus 4 as the lead agent and Claude Sonnet 4 subagents outperformed single-agent Claude Opus

4 by 90.2% on internal research evaluations. The architecture's ability to distribute work across agents with separate context windows enabled parallel reasoning that a single agent couldn't achieve.

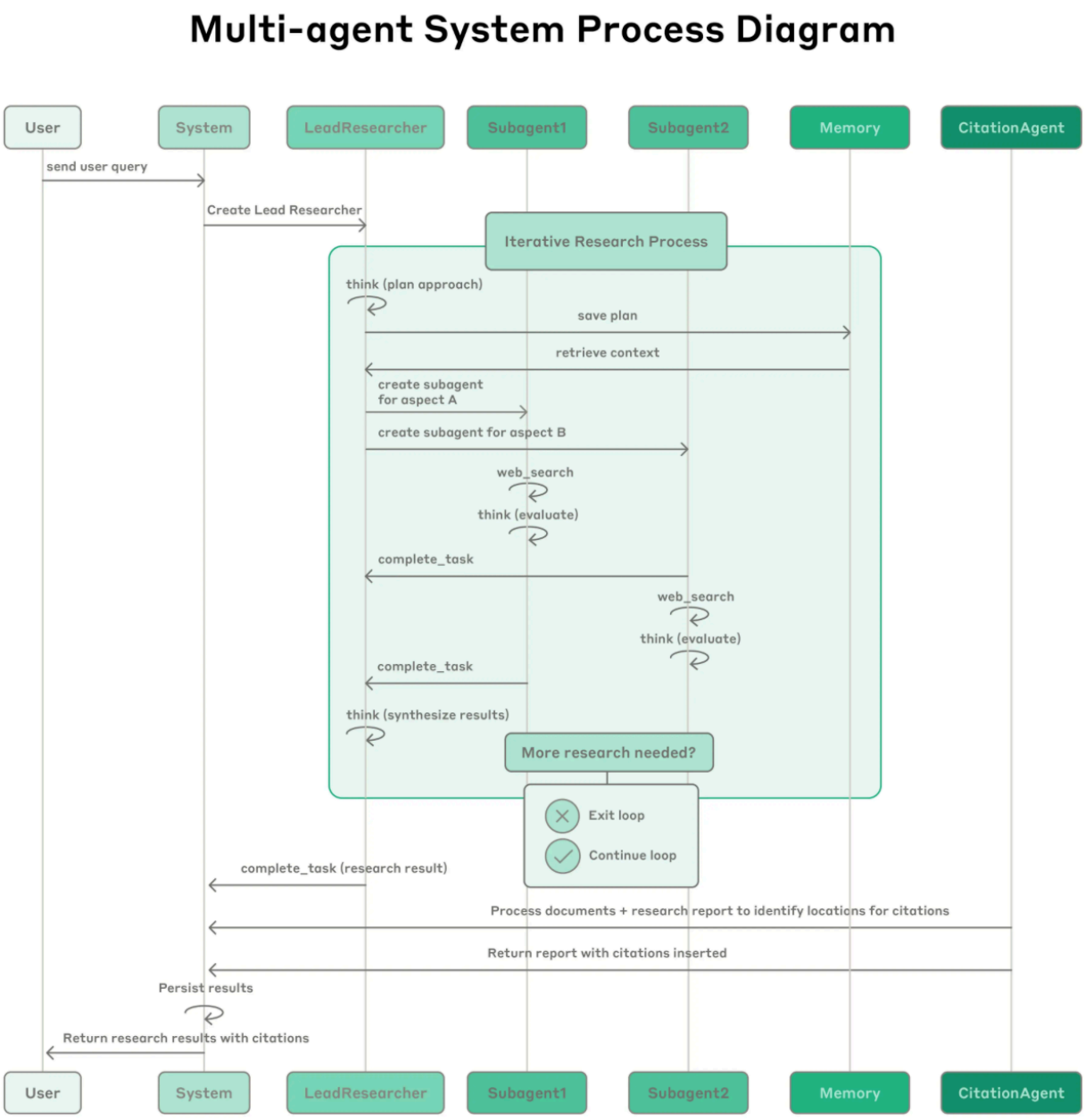


Diagram Description :

Process diagram showing the complete workflow of our multi-agent Research system. When a user submits a query, the system creates a LeadResearcher agent that enters an iterative research process. The LeadResearcher begins by thinking through the approach and saving its plan to Memory to persist the context, since if the context window exceeds 200,000 tokens it will be truncated and it is important to retain the plan. It then creates specialized Subagents (two are shown here, but it can be any number) with specific research tasks. Each Subagent independently performs web searches, evaluates tool results using interleaved thinking, and returns findings to the LeadResearcher. The

LeadResearcher synthesizes these results and decides whether more research is needed —if so, it can create additional subagents or refine its strategy. Once sufficient information is gathered, the system exits the research loop and passes all findings to a CitationAgent, which processes the documents and research report to identify specific locations for citations. This ensures all claims are properly attributed to their sources. The final research results, complete with citations, are then returned to the user.

Multi-Agent Architectures

Four architectural patterns form the foundation of most multi-agent applications: subagents, skills, handoffs, and routers. Each takes a different approach to task coordination, state management, and sequential unlocking. Below we outline a framework for selecting an architecture that best addresses your most critical constraints.

A .Subagents — *a boss that delegates*

A supervisor agent calls specialised sub-agents **as if they were tools**. The supervisor holds the conversation; the sub-agents are **stateless** (they don't remember past turns — each call starts clean). All routing passes through the boss, and it can fire several sub-agents in parallel.

- **Best for:** several distinct domains that need one coordinator (e.g. a personal assistant that spans calendar, email, and CRM), where the sub-agents don't need to talk to the user directly.
- **Main cost:** one **extra model call** per interaction, because every result must flow *back through the boss*. You pay latency and tokens for that central control and clean context.

Subagents are stateless. What if they are Stateful ?

A stateless sub-agent *does* have its own working context while it runs — it reads, thinks, searches, all inside its own context window for that one job.

Stateless only means it doesn't keep anything **between** calls. The supervisor calls it, it works, it returns a result, it forgets. Call it again later and it starts clean, with no memory of the earlier call. The session's real memory — the plan, the history, what's been done so far — lives with the *supervisor*, not the workers.

So the design question was never "should sub-agents have memory?" It's "where should the memory live?" This pattern's answer: concentrate it in one place, the supervisor. And there are four concrete reasons that's the right default.

1. One source of truth.If the supervisor holds state and the sub-agents also hold state, you now have several copies of "what's going on" that can drift apart (memory drift). When

the supervisor's picture and a sub-agent's picture disagree, who's right? Keeping state only in the supervisor means one coherent picture and nothing to reconcile.

2. Clean parallelism. The supervisor's whole advantage is firing off many sub-agents at once. Stateless workers don't interfere with each other — each takes its input, produces its output, independently. The moment sub-agents carry and share state, running them in parallel invites collisions: two of them updating the same thing, order suddenly mattering, the result depending on timing. Statelessness is what makes the fan-out safe.
3. Reusability and testing. A stateless sub-agent is a clean building block — same input gives the same kind of output, with no hidden history. You can reuse it anywhere and test it on its own. A stateful one behaves differently depending on what it happens to remember, so you can't reason about it without replaying its whole past.
4. Predictable cost. Stateless means each call costs about the same. If a sub-agent accumulated state across calls, its context would keep growing and its token cost would climb unpredictably — the exact bloat that drags down the Skills pattern from the notes.

So statelessness isn't a limitation of sub-agents. It's the property that makes the supervisor pattern work at all.

When a sub-agent genuinely *needs* memory — say a long multi-step job where it must track its own progress — the clean move, and the one Anthropic actually uses, is don't make the agent itself stateful; put the state outside it, in a shared store. The agent writes its plan and findings to a file or memory store; when context fills up, the supervisor spawns a *fresh, clean* sub-agent that reads from that store to continue. You get the memory benefit without any agent carrying hidden state.

B. Router — *split, run in parallel, merge*

A routing step reads the query, sends it to the right specialists **in parallel**, and synthesises their outputs into one answer. Routers are usually **stateless** — each request is handled fresh.

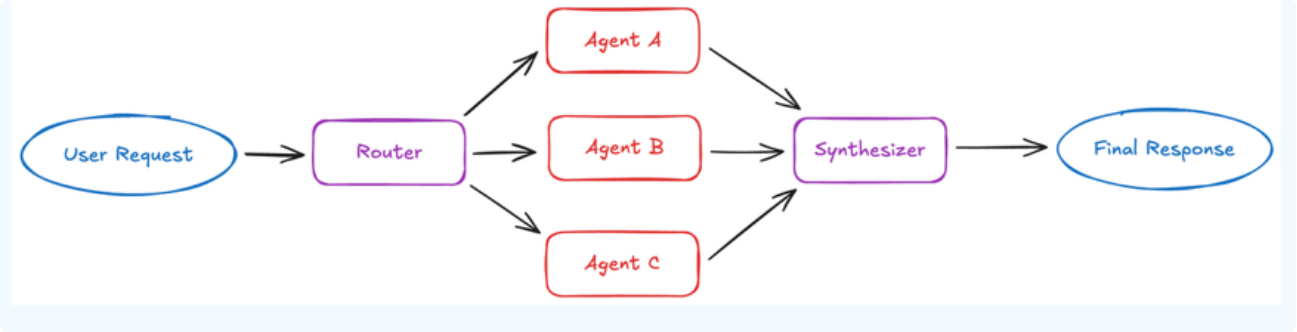
- **Best for:** distinct verticals (separate knowledge areas) where you query several sources at once and combine the results — enterprise knowledge bases, multi-vertical support.
- **This is the closest cousin of the "parallel agents" from Part 1.** Split → run at once → synthesise. Note it well: the parallelism session already taught you the engine inside the Router pattern.

(Excerpt from Langchain Blog on Subagents :)

How it works: The router decomposes the query, invokes zero or more specialized agents in parallel, and synthesizes results into a coherent response. Routers are typically stateless, handling each request independently.

Best for: Applications with distinct verticals (separate knowledge domains), scenarios requiring queries across multiple sources in parallel, or situations where you need to synthesize results from multiple agents. Examples include enterprise knowledge bases and multi-vertical customer support assistants.

Key tradeoff: Stateless design means consistent performance per request, but repeated routing overhead if you need conversation history. Can be mitigated by wrapping the router as a tool within a stateful conversational agent.



- **Router = a line.** Results move forward once → synthesise → stop. The intermediate outputs don't trigger another round.
- **Supervisor = a loop.** Results come *back* to the supervisor → the supervisor decides what to do next → it can send work out again.

Router or Subagents?

This is a sharp question, because both have a coordinator and both can run things in parallel. So the difference is *not* "one is parallel and one isn't." The real dividing line is a single question:

Is one pass enough, or do you need to look at the results and then decide what to do next?

Router is **one-pass and predictable**. The coordinator's job is *classification* — decide which of the known verticals apply, send the work out, stitch the answers together. It does not reason over what comes back and go again. You reach for it when the query decomposes cleanly into areas you already know, and a single round of retrieve-and-synthesise gives the answer. The investor question above is exactly that.

Subagents (orchestrator-worker) is **iterative and path-dependent**. The supervisor reasons over what the workers return and can *re-delegate* — spawn more workers, refine the task,

change direction based on what round one revealed. You reach for it when you can't predict the subtasks in advance, and what you find early determines what you investigate next.

Stay in mutual funds and contrast two questions to feel the switch:

"How has Fund X performed, what's it holding, and what's the exit tax?" → **Router.** Three fixed verticals, independent, one pass. The coordinator never needs to think about the answers before finishing.

"Should we launch a new small-cap fund?" → **Subagents.** This is open-ended. The lead has to explore the current small-cap landscape, competitor funds already in that space, overlap with our existing funds, regulatory constraints, investor demand — and what it learns in one area reshapes the next. If it finds the small-cap space is saturated, it pivots to digging into differentiation; if regulation is the blocker, it pivots there. You cannot pre-classify this into fixed boxes. The supervisor must read the findings and *decide the next move*, possibly several times. That reasoning loop is what makes it Subagents.

So the clean rule is : : **if the work is "sort the question into known areas, answer once, combine" → Router. If the work is "investigate, look at what came back, then decide what to do next" → Subagents.** Router classifies; the supervisor reasons. And when you're unsure, ask whether round two of the task depends on round one's results — if it does, you're in Subagents territory.

C. Skills — *load expertise only when needed*

Technically one agent, but it behaves like several. It carries many packaged specialisations (instructions + scripts + resources). At startup it knows only the **names and descriptions** of each; when one becomes relevant, it loads that skill's full detail on demand. LangChain calls this *progressive disclosure*.

- **Best for:** a single agent with many possible specialisations and no hard constraints between them — coding assistants, creative assistants.
- **Main cost:** each loaded skill piles up in the conversation history, so later calls carry more and more context — **token bloat** over a long session.

In simple terms .....

In Skills, there are **no extra agents at all.** It's one single agent, and "skills" are just **folders of instructions and reference material** that this one agent pulls in when it needs them. That's the whole idea.

**What a "skill" actually is ?

A skill is a packaged bundle — think a folder — containing specialised instructions and knowledge for one kind of task. Inside it: a set of instructions ("here's exactly how to do a SIP calculation, step by step"), maybe some reference facts, maybe a small script the agent can run. It is **not** an agent. It doesn't think or act on its own. It's material that sits on a shelf until the agent decides to read it.

Why load skills "as per the query" — the problem being solved.

Imagine your one agent needs to handle twenty different specialised tasks: SIP calculations, tax rules, KYC steps, redemption rules, and so on. The naive approach is to stuff the full instructions for all twenty into the agent's prompt up front. That fails for a real reason — the prompt gets enormous, the agent gets slower and more confused, and you're paying for all twenty sets of instructions on every single query even when the user only asked about one. That's wasteful and it degrades quality.

Skills fix this by **not** loading everything up front. Here's the actual flow:

1. At startup, the agent is given only a **table of contents** — just the *names and one-line descriptions* of all twenty skills. "SIP-calculation: computes SIP returns and schedules." "Redemption-rules: exit loads and settlement timelines." That's it. Cheap and small.
2. A user asks something — say, *"If I invest ₹10,000 monthly for 5 years, what will my SIP be worth?"*
3. The agent looks at its table of contents, recognises this matches the **SIP-calculation** skill, and **only now loads that one skill's full instructions** into its context.
4. It follows those detailed instructions to answer. The other nineteen skills were never loaded — the agent never spent context or tokens on them.

So "loaded as per the user query" means exactly this: the agent starts knowing only *what skills exist*, and pulls in the *full detail* of a skill **only when a query makes it relevant.** LangChain's name for this is *progressive disclosure* — reveal detail progressively, as needed, instead of all at once.

**The one-line contrast that will lock it in* :

In the Subagents pattern, when a specialised task comes up, the agent **calls another agent** to handle it.

In the Skills pattern, when a specialised task comes up, the **same agent loads a set of instructions** and handles it itself. Delegating to a helper versus reading a manual and doing it yourself. That's the entire difference.

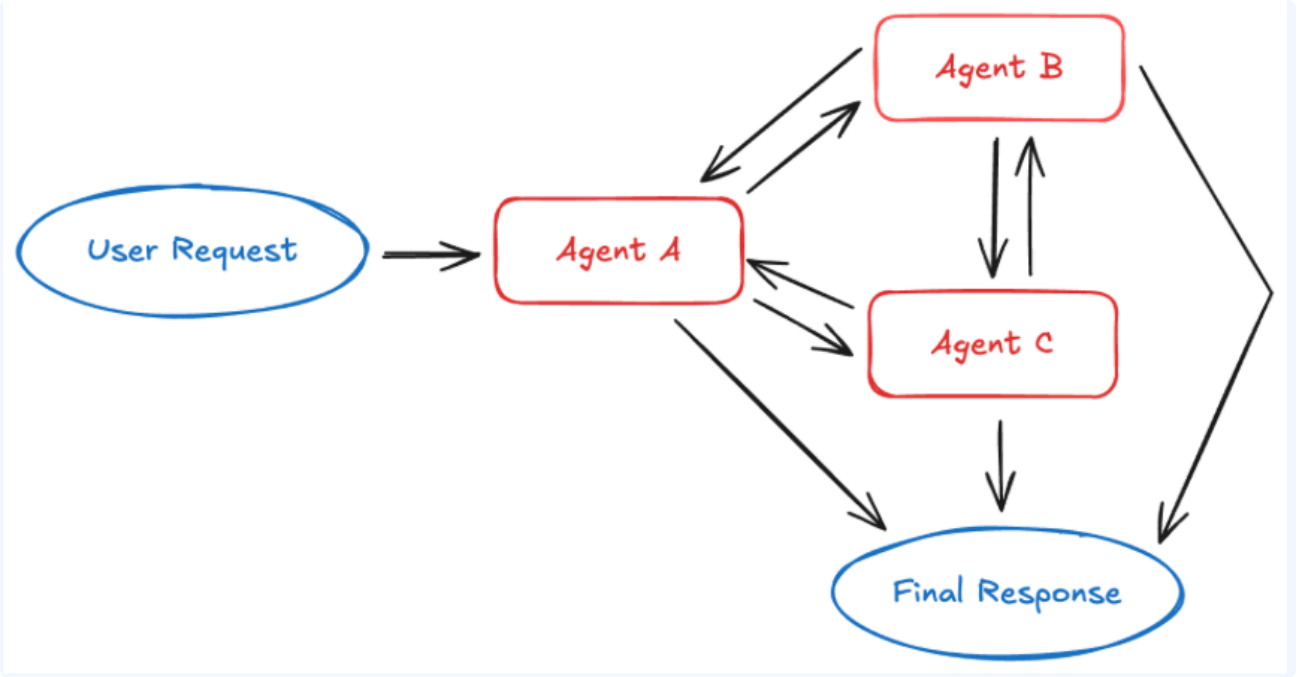
D. Handoffs — *pass control from stage to stage* (State-driven transitions)

In the handoffs pattern, the active agent changes dynamically based on conversation context. Each agent has the ability to transfer to others via tool calling.

How it works: When an agent calls a handoff tool, it updates state that determines the next agent to activate. This can mean switching to a different agent or changing the current agent's system prompt and available tools. The state survives across conversation turns, enabling sequential workflows.

Best for: Customer support flows that collect information in stages, multi-stage conversational experiences, or any scenario requiring sequential constraints where capabilities unlock only after preconditions are met.

Key tradeoff: More stateful than other patterns, requiring careful state management. However, this enables fluid multi-turn conversations where context carries forward naturally between stages.



When handoffs are useful — the tell. Reach for handoffs when a conversation moves through **stages in a fixed order**, where later stages should only open **after** earlier ones are done, and the user keeps talking to the agent the whole way through. The key word is *sequential*. If you find yourself saying "first collect this, then only after that do this, and only at the very end do that" — that's a handoff flow.

The example — a loan application assistant.

A user wants to apply for a personal loan through a chat assistant. The natural stages are: **collect details → verify eligibility → complete application.** They must happen in that order — you can't check eligibility before you have the income figure, and you can't submit before eligibility passes. That ordering is exactly what handoffs enforce.

Here's the flow, stage by stage. Notice the **active agent changes** as the conversation advances:

1. **Intake agent** is active first. Its whole job is to collect the required details — name, income, loan amount, tenure. Its instructions and tools are scoped to *just that*. It talks to the user, gathers the fields, and once it has them all, it **hands off** to the next stage.
2. The handoff **updates the shared state** — it records "intake complete, here are the collected details" — and that state decides who becomes active next. Now the **eligibility agent** is active. It reads the details already collected (that's why state carrying forward matters), checks them against the lending rules, and decides pass or fail. If it passes, it **hands off** again.
3. The **application agent** becomes active. It takes everything gathered so far and walks the user through finalising and submitting.

Two things to see in that flow. First, **the user is talking to "the assistant" the entire time** — from their side it feels like one continuous conversation, even though control passed across three different agents underneath. Second, **the state survives the handoffs** — the income the intake agent collected is still there when the application agent needs it. That carried-forward memory is what makes handoffs *stateful*, and it's why this pattern is different from Router or Subagents, which start each request clean.

Requirement → pattern, at a glance

What you actually need	Reach for
Several distinct domains, one coordinator, want parallel work	**Subagents**
One agent, many optional specialisations, lightweight	**Skills**
A staged flow where the agent keeps talking to the user and stages unlock in order	**Handoffs**
Separate verticals, query many sources in parallel, then synthesise	**Router**